

Lucius Hatherly Beer Quality Kaggle Report:

Kaggle Team Name: ift6390_Lucius Hatherly_20294880

Overview: This report presents my approach to predict beer quality based on the fermentation and chemical properties. This project utilizes feature engineering, preprocessing, and machine learning optimization. Note the models explored in the reports generates two main sets of submission, one prior to October 25th and one after October 25th.

1. Feature Preprocessing (Feature Design):

Data Cleaning & Preparation: The dataset contained categorical and numeric features which described the chemical composition and brewing process of beer. The target variable we predicted was quality, which is ordinal and can be given a value between 0 to 10.

Initial Data Cleaning & preprocessing steps involved:

- **Handling Missing Values:** Missing values in the numeric columns were filled in with median values and categorical columns were filled in with mode. This ensured that post scaling, the learning algorithms executed complete data matrices without introducing bias problems or reducing the sample size.
- **Outlier Removal:** Removal of outlier values was performed to reduce overfitting and improve model's robustness. This specifically involved removing all values below the 5th percentile and above the 95th percentile for the free_CO2, dissolved_oxygen, and final_gravity columns, and removed all values below the 1st percentile and above the 99th percentile for the alcohol_ABV and fermentation_strength columns, and lastly all other columns had values below the 2nd percentile and above 98th percentile removed.
- **Label Encoding:** I applied Label Encoding for the categorical variable (beer-style) given the column is non-continuous and ordinal. Label Encoding was performed as Machine Learning algorithms mostly need numeric columns as inputs to work. One Hot Encoding was avoided as it increases dimensionality and cannot capture the ordinality of the beer style.

Note that for the submission prior to October 25th, I performed handling missing values, outlier removals, and label encoding. However, meaningful feature engineering and selection including removing and adding columns were only conducted for the submission after October 25th. This is because the goal of the submissions prior to October 25th was focused on beating the random and strong Kaggle baseline rather than refined feature engineering and optimizing the model as much as possible like the post October 25th submissions.

Feature Selection and Engineering: Conducting correlation analysis as seen in the Appendix Figure D revealed that: 1. alcohol_ABV and fermentation_strength were more correlated with quality (~0.43 and 0.41 respectively), and these features were also strongly correlated with one another (~0.95, see Appendix Figure O), 2. Variables like pH, final_gravity, and dissolved_oxygen had weak correlations (<0.05), see Figure D again, and were candidates for removal. Moreover, I then removed the pH and fermentation_strength columns, as pH had a weak correlation with quality, and fermentation_strength was irrelevant to predicting quality given alcohol_ABV.

After several iterations, we determined that removing low-correlation columns (pH, fermentation_strength) for the models improved validation accuracy for Kaggle from around 36% for pre-October 25th without meaningful feature engineering compared to around 38% post October 25th for the most optimal models. Interestingly, the Python validation dropped slightly from pre-October 25th to post-October 25th for the highest

performing models. Overall, the only slight changes in Kaggle accuracy and similar Python validation scores, supports that the pre-October 25th evaluated models already largely captured the key non-linear interactions among the original features.

For a final test, I decided to apply more feature engineering than just dropping the redundant columns. I also created a new column and evaluated these enhanced features for the highest performing model. Specifically, given the strong correlation between alcohol_ABV and fermentation_strength. I decided to add a new feature, alcohol_efficiency, ($\text{alcohol_efficiency} = \text{alcohol_ABV} / (\text{fermentation_strength} + 10^{-6})$). Note, this feature captured the relative efficiency of fermentation in synthesizing alcohol which may reflect brewing yield differences. Interestingly, this change led to the highest Kaggle classification performance of more than 39%, although the Python validation stayed around 61%.

2. Methodology

Data Splitting: For both the pre and post October 25th model testing, I divided the dataset into training (80%) and validation (20%) subsets and utilized stratified sampling to ensure that the quality score was properly maintained across the data set.

Model Selection: Pre-October 25th, I evaluated a Logistic Regression and Random Forest Model on the data without any meaningful hyperparameter tuning. Both methods were successful in passing the Kaggle baseline. For post-October 25th model selection, I trained different supervised learning models and compared these models using consistent preprocessing pipelines. These models included: Logistic Regression (multinomial, regularized), Linear SVM, k-Nearest Neighbors, Gaussian, Complement Naïve Bayes, and Random Forest Classifier. These models were selected to show various learning paradigms – non-parametric, ensemble, linear, and probabilistic methods – enabling a comprehensive comparison of how these different algorithms biases process chemical and other patterns for beer quality. KNN and Random Forests were added to better capture non-linear interactions and correlation between features predicting quality compared to a baseline of linear models like Linear SVM, Logistic Regression, etc, which could ideally capture key linear interactions. More complicated networks like Gradient Boosting and Neural Networks were excluded from being tested due to Kaggle rules.

Hyperparameter Optimization: For these post October 25th models, I then performed hyperparameter optimization by utilizing a 5-fold Stratified Cross-Validation Grid Search applied to each model individually. To give you example, the hyper parameter grid investigated for Random Forest (the most successful model), initially spanned: $n_estimators \in [180, 200]$, $max_depth \in [15, 20]$, $min_samples_split \in [3, 4]$, $min_samples_leaf \in [1, 2]$, $max_features = 'sqrt'$, $class_weight = 'balanced_subsample'$. These tuning processes helped to maximize validation accuracy under class-imbalance constraints.

Regularization & Scaling: These were the regularization and scaling I employed for my models: 1) Linear models used L2 regularization and StandardScaler, 2) Distance-based models (KNN) also used standard scaling, 3) Naïve Bayes models used Min-Max scaling for numeric stability, and 4) Random Forests, while tree-based, were wrapped in pipelines with standard scalers for consistency across folds. Standard Scalers help assume a Gaussian distribution, centers data around zero, and is less sensitive to outliers. Min-Max scaling helps preserve the original shape of the distribution and is useful for bounded inputs.

Model Evaluation, Performance Metrics, and Error Analysis: After performing the training of the post October 25th models, we evaluated the best model across the hyperparameter options for each classifier. For Performance metrics, we calculated accuracy, F1, precision, recall, and support for both the training and

validation sets. Then we analyzed error metrics for both the validation and test sets using metrics including mean absolute error (MAE), root mean squared error (RMSE), and general error (1-Accuracy). Subsequently, we selected the model with the best accuracy and lowest error rate for further hyperparameter tuning to obtain the best features possible and used this to generate the csv files we submitted for Kaggle predictions. Finally, as mentioned earlier, I performed a last experiment by adding a new feature, alcohol efficiency, and dropping the pH and fermentation features directly on the train and test set (while avoiding leakage) rather than before the creation of the train and test set (which I performed originally), and then evaluated the transformed data with a Random Forest model (with hyperparameters similar to the highest performing model).

3. Results

Post October 25th Validation set performance for Random Forest:
Accuracy: 0.6119

	precision	recall	f1-score	support
3	0.00	0.00	0.00	4
4	0.00	0.00	0.00	23
5	0.65	0.65	0.65	299
6	0.59	0.76	0.66	397
7	0.61	0.34	0.44	151
8	1.00	0.05	0.10	19
9	0.00	0.00	0.00	1
accuracy			0.61	894
macro avg	0.41	0.26	0.26	894
weighted avg	0.60	0.61	0.59	894

Figure A (Performance Metrics for Random Forest)

Post October 25th Validation Set Model Performance Summary:

	Model	Accuracy
2	Random Forest	0.611857
3	KNN	0.573826
0	Logistic Regression	0.563758
1	Linear SVM	0.527964
4	Gaussian Naive Bayes	0.514541
5	Complement Naive Bayes	0.460850

Figure B (Model Performance across training and validation)

```
{'clf_bootstrap': True, 'clf_class_weight': 'balanced_subsample', 'clf_max_depth': 30, 'clf_max_features': 'sqrt', 'clf_min_samples_leaf': 2, 'clf_min_samples_split': 2, 'clf_n_estimators': 400}
```

Best CV Accuracy: 0.6193

Validation Accuracy (Best RF): 0.6174

(Figure C: Best parameters found for Random Forest after Additional Hyperparameter Investigation)

The Random Forest model showed a strong predictive performance on the validation set compared to all other classifiers as seen with Figure B. Within Figure C, Random Forest achieved a validation accuracy of 0.6174, and Figure A shows the Random Forest model having a balanced precision-recall trade-off (weighted average F1 $\approx$ 0.59). The appendix's Figure E's confusion matrix indicates that most misclassifications occur between adjacent quality classes (e.g., predicting 5 as 6 or vice versa), which could be interpreted due to the target classes being ordinal and that 5 and 6 are the most common columns. Specifically, we should note that low-frequency classes (quality 3, 4, 8, and 9) show minimal recall, reflecting class imbalance in the dataset. Classes 1 and 2 do not exist in the confusion matrix due to their absence from training data, further reflecting the class imbalance issues. Figure B demonstrated that Random Forest outperforms KNN, SVM, Logistic Regression and Naive Bayes on the validation set, with the highest accuracy (0.6118 – before further hyperparameter testing as shown in Figure C), and lowest error measures across both training and validation folds (as seen in Figure M in the Appendix). The difference we can see between training accuracy - 1.0 and validation accuracy - 0.6119 – both values found in the Appendix Figure M implies some overfitting. However, Random Forest having a better generalization compared to other models implies this is an acceptable tradeoff. Moreover, Feature importance results (Appendix Figure F) reveal that alcohol_ABV, lactic_acid, diacetyl_concentration, and free_CO2 are the key predictors of beer quality. These features match the

chemical characteristics that influence taste, carbonation and fermentation standards. All of this aligns well with the commonsense intuition that these features are key to brewing importance. Oppositely, `beer_style` and `original_gravity` contributed the smallest amount, this could imply that preliminary density attributes and stylistic preferences are comparatively unimportant when alcohol and acidity levels are considered.

In summary, the Random Forest model captures interactions and is a better generalizer on unseen data in contrast to simpler classifiers like KNN. Random Forest further generates more interpretable insights for the most influential brewing parameters driving perceived beer quality.

4. Discussion:

Pre October 25th: The Logistic Regression Model and Random Forest Model have a Python validation complexity of 57.4% and 61.9%, along with a Kaggle complexity of around 31.8 to 32% and 36.9% respectively. These methods both beat the Kaggle random and strong baseline. However, given that these methods do not conduct hyperparameter tuning, serious feature engineering, and error analysis, it is highly likely that these models could be improved with these methods being implemented. Note, finding and improving these models which beat the baseline was the key focus of the post October 25th model investigation.

Post October 25th: The optimized Random Forest model (before was the most reliable, reaching a validation accuracy of around 61.8% and a Kaggle accuracy of 38%. When we consider the ordinal and relatively imbalanced nature of the quality variable, the classifier's performance is strong relative to baseline and simpler classifiers like KNN and Logistic Regression. The Random Forest model can be seen as managing collinearity, non-linear feature relationships, and interactions between features without needing to have explicit polynomial expansions. Interestingly though, my model where I performed additional feature engineering directly on the train and validation set had a Python accuracy of around 61.7% (see Appendix Figure N), but a higher Kaggle accuracy of 39.15%. The possibility for these better Kaggle results is because of the advanced feature engineering and using the existing random forest model.

Several key advantages of the Random Forest model include having good interpretability of the model through having analyzed feature significance which aligns intuitively with the real brewing process. The model further employs cross validated hyperparameter tuning and an end-to-end standardized pipeline which minimizes data leakage. Lastly, the Random Forest displays ensemble robustness to feature redundancy and non-linearities. However, limitations of the model which minimizes the accuracy of the results include class imbalances with few high-quality and low-quality beers, along with an overfitting of the Random Forest and the other evaluated models in general. In addition, chemical data for the beer could omit latent production and sensory variables which impact quality. Significantly, the minimal benefit from newly engineered or removal of redundant features could suggest diminishing improvements in model accuracy, and further improvements with the Random Forest model being unlikely to augment accuracy and Kaggle performance. To improve my analysis further, I could widen my model experimentation to experiment with stacked ensembles, neural networks, or gradient boosting models for finer variance control. These more complex models could help merge rare quality labels to reduce output granularity and stabilize predictions along with improving overall accuracy past 45% on Kaggle.

In conclusion, the Random Forest classifier is the best performing model for this dataset. It balanced accuracy, interpretability, and generalization. Overall, the study demonstrates that ensemble tree-based models are a robust choice for structured, chemistry-driven prediction tasks for beer quality.

5. Appendix:

Top Correlated Features:

quality	1.000000
alcohol_ABV	0.432202
fermentation_strength	0.414471
lactic_acid	0.089932
free_CO2	0.074866
gypsum_level	0.057630
pH	0.010090
final_gravity	-0.020877
dissolved_oxygen	-0.035196
original_gravity	-0.071395
bitterness_IBU	-0.072348
sodium	-0.209092
diacetyl_concentration	-0.253498

Name: quality, dtype: float64

Figure D (Correlation Matrix showing relationship for all feature columns with quality)

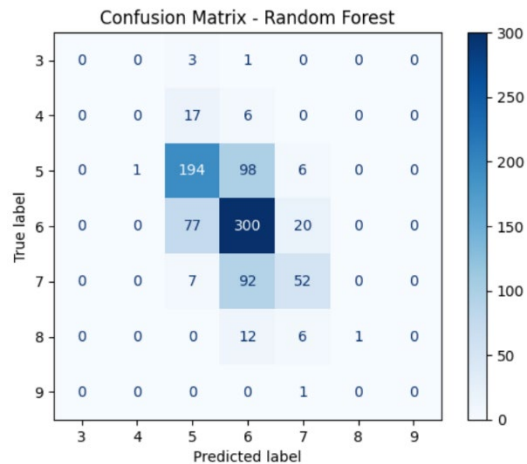


Figure E (Confusion Matrix for Random Forest)

	Feature	Significance
10	alcohol_ABV	0.147260
3	lactic_acid	0.109621
2	diacetyl_concentration	0.097600
6	free_CO2	0.094180
1	bitterness_IBU	0.087656
5	sodium	0.085773
4	final_gravity	0.084076
7	dissolved_oxygen	0.074220
9	gypsum_level	0.073951
11	alcohol_efficiency	0.070047
8	original_gravity	0.055028
0	beer_style	0.020589

Figure F (Feature Significance for Random Forest)

```
## Splitting Data into Train and Test Validation Sets:
from sklearn.model_selection import train_test_split
# Defining training and validation data

# Importing Label Encoder
le = LabelEncoder()
# Applying label encoding fit transform on beer style for training
df_train['beer_style'] = le.fit_transform(df_train['beer_style'])
# Applying label encoding transform to beer style
df_test['beer_style'] = le.transform(df_test['beer_style'])

rng = np.random.RandomState(2)

# Removing quality column from X as quality is the label
X = df_train.drop(columns="quality")
# Assigning quality to y
y = df_train["quality"]

# Using train test split to create training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Figure G (Code showing train test split of data)

```
# Defining Random Forest Hyperparameters
# Including n_estimators, max_depth, min_samples_split,
"Random Forest": {
  "clf_n_estimators": [180, 200],
  "clf_max_depth": [15, 20],
  "clf_min_samples_split": [3, 4],
  "clf_min_samples_leaf": [1, 2],
  "clf_max_features": ["sqrt"],
  "clf_bootstrap": [True],
  "clf_class_weight": ["balanced_subsample"]
},
```

Figure H (Code showing initial hyper parameters searched for Random Forest)

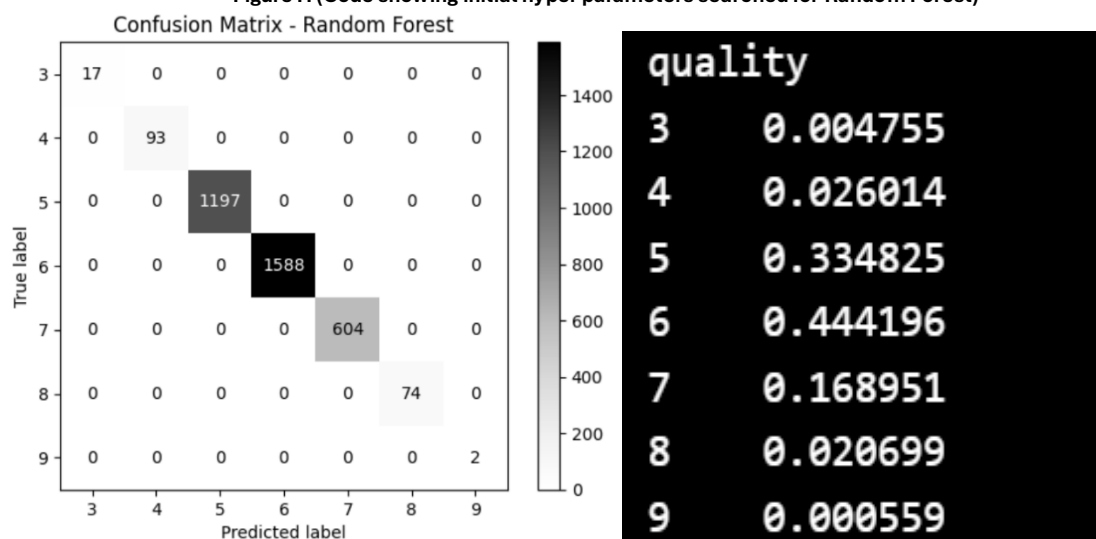


Figure I (Confusion Matrix for Training Set of Random Forest)

Figure J (Distribution of quality values)

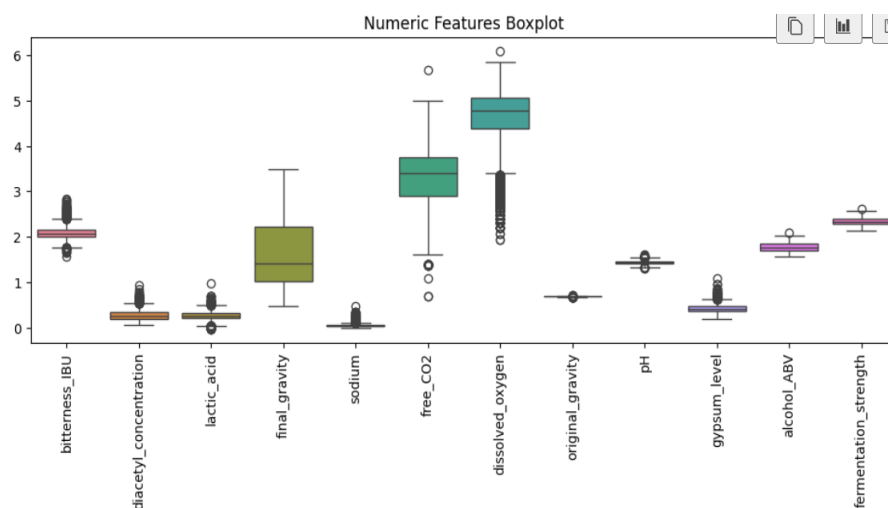


Figure K (Boxplot for Numeric Features of Data)

```

Investigating the missing values before imputation.
beer_style          0
bitterness_IBU      0
diacetyl_concentration  0
lactic_acid         0
final_gravity       0
sodium              0
free_CO2            0
dissolved_oxygen    0
original_gravity    0
pH                  0
gypsum_level        0
alcohol_ABV         0
fermentation_strength 0
quality             0
dtype: int64

Missing values after imputations for Training data:
0

Missing values after imputations for Testing data:
0

```

Figure L (Table showing results of data imputation)

Summary of Errors across Models Post October 25th:						
	Model	Train Accuracy	Train Error	Train MAE	Train RMSE	\
2	Random Forest	1.000000	0.000000	0.000000	0.000000	
3	KNN	1.000000	0.000000	0.000000	0.000000	
0	Logistic Regression	0.584615	0.415385	0.457063	0.739306	
1	Linear SVM	0.557483	0.442517	0.487273	0.762403	
4	Gaussian Naive Bayes	0.495105	0.504895	0.594406	0.904534	
5	Complement Naive Bayes	0.463217	0.536783	0.647832	0.948167	
	Accuracy Validation	Error Validation	MAE Validation	RMSE Validation		
2	0.611857	0.388143	0.429530	0.717315		
3	0.573826	0.426174	0.486577	0.783641		
0	0.563758	0.436242	0.485459	0.771413		
1	0.527964	0.472036	0.524609	0.801983		
4	0.514541	0.485459	0.585011	0.911645		
5	0.460850	0.539150	0.656600	0.952449		

Figure M (Error Metrics for various models)

Evaluating the Random Forest Model with feature engineering post train test split:
Accuracy is: 0.6174496644295302

Classification Report:

	precision	recall	f1-score	support
3	0.00	0.00	0.00	4
4	0.50	0.04	0.08	23
5	0.65	0.68	0.66	299
6	0.61	0.72	0.66	397
7	0.56	0.40	0.47	151
8	1.00	0.11	0.19	19
9	0.00	0.00	0.00	1
accuracy			0.62	894
macro avg	0.47	0.28	0.29	894
weighted avg	0.62	0.62	0.60	894

Figure N (Results of Feature Engineering for Random Forest model post train-test split)

	alcohol_ABV	fermentation_strength
alcohol_ABV	1.000000	0.948021
fermentation_strength	0.948021	1.000000

Figure O (Correlation between alcohol_ABV and fermentation_strength)

Validation Accuracy of Logistic Regression Model pre-October 25th: 0.5738255033557047

Classification Report:

	precision	recall	f1-score	support
3	0.00	0.00	0.00	4
4	0.00	0.00	0.00	23
5	0.62	0.64	0.63	299
6	0.57	0.71	0.63	397
7	0.48	0.26	0.34	151
8	0.00	0.00	0.00	19
9	0.00	0.00	0.00	1
accuracy			0.57	894
macro avg	0.24	0.23	0.23	894
weighted avg	0.54	0.57	0.55	894

Figure P (Validation Accuracy of Logistic Regression Model pre October 25th)

Validation Accuracy of Random Forest Pre October 25th: 0.6185682326621924

Classification Report (Validation Set):

	precision	recall	f1-score	support
3	0.00	0.00	0.00	4
4	0.50	0.04	0.08	23
5	0.66	0.70	0.68	299
6	0.61	0.71	0.66	397
7	0.54	0.40	0.46	151
8	1.00	0.05	0.10	19
9	0.00	0.00	0.00	1
accuracy			0.62	894
macro avg	0.47	0.27	0.28	894
weighted avg	0.62	0.62	0.60	894

Figure Q (Validation Accuracy of Random Forest before October 25th)



submission_logistic_regression.csv

0.31764



Complete · 17d ago

Figure P (Kaggle Logistic Regression Submission accuracy before October 25th beating baseline)



submission_random_forest.csv

0.36862



Complete · 18d ago · Random Forest

Figure Q (Kaggle Random Forest Submission accuracy before October 25th beating baseline)



submission_best_random_forest.csv

0.38039



Complete · now

Figure R (Kaggle Random Forest with optimal hyperparameters)



Post_October_25_Random_Forest_AFE.csv

0.39215



Complete · 1m ago

Figure S (Kaggle Random Forest Model with additional Feature Engineering)